Рубежный контроль № 1
по курсу «ПиК ЯП»

Вариант запросов: В Вариант предметной области: 2
Предметная область: «Школьник — Класс»

Цель работы

Разработать программу на Python с классами данных и связями «один-ко-многим» и «многие-ко-многим», реализовать три запроса варианта В.

Описание классов и запросов

Класс Pupil содержит идентификатор, фамилию школьника, учебный балл (количественный признак вместо зарплаты из примера методички) и идентификатор класса. Класс SchoolClass содержит идентификатор и название класса. Промежуточный класс PupilClass реализует связь «многие-ко-многим».

В1. Вывести список всех школьников, у которых фамилия начинается с буквы «А», и названия их классов (отношение «один-ко-многим»).

В2. Вывести список классов с минимальным баллом школьников в каждом классе, отсортированный по минимальному баллу по возрастанию (отношение «один-ко-многим»).

В3. Для связи «многие-ко-многим» вывести список всех связанных школьников и классов, отсортированный по школьникам (по алфавиту фамилий).

Текст программы

```
"""Рубежный контроль №1. Вариант запросов В, предметная область 2."""
from operator import itemgetter


class Pupil:
    """Школьник: учебный балл / рейтинг."""
    def __init__(self, id, fio, score, class_id):
        self.id = id
        self.fio = fio
        self.score = score
        self.class_id = class_id


class SchoolClass:
    """Школьный класс."""
    def __init__(self, id, name):
        self.id = id
        self.name = name


class PupilClass:
    """Связь многие-ко-многим между школьниками и классами."""
    def __init__(self, class_id, pupil_id):
        self.class_id = class_id
```

```
        self.pupil_id = pupil_id


classes = [
    SchoolClass(1, '10-А математический'),
    SchoolClass(2, '11-Б гуманитарный'),
    SchoolClass(3, '9-В естественно-научный'),
    SchoolClass(4, '8-А инженерный'),
    SchoolClass(5, '11-А физико-математический'),
]

pupils = [
    Pupil(1, 'Артамонов', 88, 1),
    Pupil(2, 'Андреев', 94, 1),
    Pupil(3, 'Петров', 76, 2),
    Pupil(4, 'Аксёнов', 82, 2),
    Pupil(5, 'Иванов', 69, 3),
    Pupil(6, 'Смирнова', 91, 3),
    Pupil(7, 'Алексеева', 85, 4),
    Pupil(8, 'Васильев', 74, 5),
]

pupils_classes = [
    PupilClass(1, 1),
    PupilClass(1, 2),
    PupilClass(2, 3),
    PupilClass(2, 4),
    PupilClass(3, 1),
    PupilClass(3, 5),
    PupilClass(4, 7),
    PupilClass(5, 8),
    PupilClass(5, 2),
]


def main():
    # Соединение один-ко-многим
    one_to_many = [
        (p.fio, p.score, c.name)
        for c in classes
        for p in pupils
        if p.class_id == c.id
    ]

    # Соединение многие-ко-многим через промежуточный класс
    many_to_many_temp = [
        (c.name, pc.class_id, pc.pupil_id)
        for c in classes
        for pc in pupils_classes
        if c.id == pc.class_id
    ]

    many_to_many = [
        (p.fio, p.score, class_name)
        for class_name, _, pupil_id in many_to_many_temp
        for p in pupils
        if p.id == pupil_id
    ]

    print('Задание B1')
    # Школьники с фамилией на «А» и их классы
    res_1 = [
```

```
        (p_fio, c_name)
        for p_fio, _, c_name in one_to_many
        if p_fio.startswith('A')
    ]
    print(res_1)

    print('\nЗадание B2')
    # Классы с минимальным баллом школьников в каждом классе
    res_2_unsorted = []
    for c in classes:
        c_pupils = list(filter(lambda x: x[2] == c.name, one_to_many))
        if len(c_pupils) > 0:
            c_scores = [score for _, score, _ in c_pupils]
            min_score = min(c_scores)
            res_2_unsorted.append((c.name, min_score))

    res_2 = sorted(res_2_unsorted, key=itemgetter(1))
    print(res_2)

    print('\nЗадание B3')
    # Связанные школьники и классы, отсортированные по школьникам
    res_3 = sorted(many_to_many, key=itemgetter(0))
    print(res_3)


if __name__ == '__main__':
    main()
```

Результаты выполнения программы

Задание B1

```
[('Артамонов', '10-А математический'), ('Андреев', '10-А математический'),
 ('Аксёнов', '11-Б гуманитарный'), ('Алексеева', '8-А инженерный')]
```

Задание B2

```
[('9-В естественно-научный', 69), ('11-А физико-математический', 74),
 ('11-Б гуманитарный', 76), ('8-А инженерный', 85),
 ('10-А математический', 88)]
```

Задание B3

```
[('Аксёнов', 82, '11-Б гуманитарный'), ('Алексеева', 85, '8-А инженерный'),
 ('Андреев', 94, '10-А математический'), ('Андреев', 94, '11-А физико-математический'),
 ('Артамонов', 88, '10-А математический'), ('Артамонов', 88, '9-В естественно-научный'),
 ('Васильев', 74, '11-А физико-математический'), ('Иванов', 69, '9-В естественно-научный'),
 ('Петров', 76, '11-Б гуманитарный')]
```

Вывод

Созданы три класса данных, подготовлены тестовые наборы и реализованы связи «один-ко-многим» и «многие-ко-многим». Запросы B1, B2 и B3 успешно выполнены с использованием списковых включений, функций высшего порядка filter, sorted и модуля operator; корректные результаты получены при запуске программы Python.